

# AI多Agent编排系统 - 优化架构设计与实现

## 架构概览

本项目基于 Python 实现一个多 Agent 协同工作的编排系统，利用 **Kiro CLI**、**Beads** 和 **Agent SOP** 等工具来组织 AI 代理的工作流程。优化后的架构将不同功能模块清晰分离，增强了系统的稳定性和可扩展性。

**核心思想：** 由一个 Python 编写的 **orchestrator（调度器）** 统一管理多个 AI Agent，每个 Agent 拥有独立的 SOP（标准操作流程）文件和技能（Skill）模块，以及各自关联的任务数据。调度器通过 Python SDK（而非手工终端命令）调用 Agent，串行或并发地调度任务执行，并使用 Beads 作为任务追踪与记忆机制，确保任务可以中断恢复、递归创建子任务等。

- **Agent SOP：** 每个 Agent 的 SOP 定义了其执行特定任务的步骤流程，采用 Markdown 编写，包含任务目标、步骤和约束等信息 <sup>1</sup>。SOP 相当于为 AI 代理设计的操作手册，使复杂的多步任务在不同 Agent 和团队之间可重复复用。
- **技能 (Skill)：** 每个 Agent 具有一组技能模块，封装了该 Agent 可用的工具或操作（例如读取/写入文件、执行 Shell 命令等）。通过将技能与 Agent 解耦，每个 Agent 可以灵活定制其权限和功能，提升安全性和可控性。
- **调度器 (Orchestrator)：** 作为系统的大脑，负责从任务列表中提取待办任务，分配给合适的 Agent 执行。调度器实现了稳定的管理逻辑，可根据配置选择串行或并发方式运行多个任务，并能够协调多个 Agent 间的协同（例如一个任务完成后触发另一 Agent 执行后续任务）。
- **Beads 任务管理：** 本系统使用 Beads 作为任务的持久化存储和依赖管理工具 <sup>2</sup>。Beads 将所有任务组织为带优先级和依赖关系的有向无环图 (DAG)，并通过 Git 将任务状态持久化存储在项目的 `.beads/issues.jsonl` 中 <sup>3</sup>。每条任务（又称 bead）都有唯一 ID（如 `bd-123`），状态（`open/in_progress/blocked/closed` 等）以及依赖情况。Beads 提供了 `bd` CLI 工具，支持使用 `--json` 输出任务数据，方便我们的调度器程序解析和利用 <sup>4</sup>。

通过上述架构，系统实现了以下优化目标：

- **清晰的模块分工：** 每个 Agent 的 SOP、技能代码和任务数据彼此独立，存放于专属目录，便于团队协作和日后维护扩展。
- **稳定可控的调度：** 调度器集中管理所有 Agent 的任务执行，确保执行顺序和并发度在掌控之中。调度器设计了串行和并发两种调度模式，可配置线程池大小控制并发 Agent 数量，支持多 Agent 同时运行。
- **规范的 SOP 编写：** 约定了 SOP 文档的书写格式和规范（详见下文），包括统一的章节结构和约束标记，支持模块化和模板化生成 SOP，从而方便版本管理和知识复用。
- **任务中断恢复与递归调用：** 借助 Beads 的任务跟踪，未完成的任务状态会持久保存，当系统重启后调度器可通过 `bd ready` 自动检索待办任务继续执行。 <sup>5</sup> <sup>6</sup> 同时，Agent 在执行过程中若发现新任务需求，可动态创建子任务并以依赖关系链接到当前任务（使用 Beads 的 `discovered-from` 依赖类型 <sup>7</sup>），调度器将识别这些关系，实现任务的递归调度和闭环管理。

接下来，我们将详细介绍项目的目录结构、核心模块的实现代码，以及使用方法和开发规范。

## 项目目录结构

```
ai_multi_agent_project/      # 项目根目录
├── orchestrator.py          # 调度器主程序
```

|                         |                                 |
|-------------------------|---------------------------------|
| ├── agents/             | # 所有 Agent 定义的模块目录              |
| │   ├── __init__.py     |                                 |
| │   ├── base_agent.py   | # Agent 基类定义                    |
| │   ├── code_agent/     | # 示例 Agent: 代码编写助手              |
| │   │   ├── __init__.py |                                 |
| │   │   ├── sop.md      | # Code Agent 的 SOP 文件 (标准操作流程)  |
| │   │   ├── skills.py   | # Code Agent 的技能函数定义            |
| │   ├── design_agent/   | # 示例 Agent: 设计/规划助手             |
| │   │   ├── __init__.py |                                 |
| │   │   ├── sop.md      | # Design Agent 的 SOP 文件         |
| │   │   ├── skills.py   | # Design Agent 的技能函数定义          |
| ├── .beads/             | # Beads 任务存储目录 (Git仓库, 用于任务持久化) |
| │   └── issues.jsonl    | # Beads 任务列表主文件 (JSON Lines格式)  |

### 目录说明:

- `orchestrator.py`: 核心调度脚本, 负责读取任务、分配 Agent、执行任务及更新任务状态。
- `agents/base_agent.py`: 定义所有具体 Agent 的基础类, 封装公共逻辑, 例如加载SOP、执行任务的接口等。
- `agents/code_agent/` 和 `agents/design_agent/`: 两个示例 Agent 的模块目录, 每个Agent模块下都有:
  - `sop.md`: 该 Agent 的 SOP 文件, 采用 Markdown 编写, 定义了此Agent执行任务的标准流程。
  - `skills.py`: 该 Agent 可用的技能函数, 通常是对底层工具的封装。例如 Code Agent 的技能可能包括读取/写入代码文件、运行测试用例等; Design Agent 则可能有读取文档、生成规划报告等技能函数。
  - (可选) `__init__.py`: 使目录成为Python包, 可在 `orchestrator` 中通过 `import agents.code_agent` 引用。
- `.beads/`: Beads的隐藏目录, 存放任务数据。 `issues.jsonl` 是主要的任务数据库文件, 每行一个JSON, 记录所有任务的详情 (ID、标题、描述、状态、优先级、依赖等) <sup>3</sup>。这个文件由 Beads CLI 自动维护, 通常不手动编辑。

整个项目遵循常规的 Python 项目命名规范, 模块命名使用下划线和小写字母, 目录层次清晰, 方便进一步扩展新的 Agent 模块或其他功能。

## 核心模块与实现

以下将介绍本项目的核心代码, 包括调度器和示例 Agent 的实现。所有代码均以代码块形式给出完整内容, 并附带注释以帮助理解。

### 调度器: `orchestrator.py`

调度器是系统的控制中枢。它通过周期性查询 Beads 获取当前所有“已就绪” (ready) 的任务列表, 然后按照预定策略将任务交由相应的 Agent 去执行。调度器确保任务执行的可控性: 可根据需要串行地执行任务 (一个一个依次完成), 或并发执行多个任务。同时, 它负责在任务完成后更新任务状态 (如关闭已完成的任务), 并处理新发现的子任务。

下面是 `orchestrator.py` 的完整代码实现:

```
python
import os
import json
```

```

import subprocess
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import List, Dict, Any

# 配置：最大并发执行的 Agent 数量（并发线程数）。设置为1则串行执行。
MAX_CONCURRENT_AGENTS = 2

# 定义任务类型与Agent的映射。如无特定类型匹配则使用"default"对应的Agent。
AGENT_REGISTRY: Dict[str, 'BaseAgent'] = {}

# 引入我们定义的 Agent 模块
from agents.base_agent import BaseAgent
from agents import code_agent, design_agent

# 初始化 Agent 实例并注册到映射表
AGENT_REGISTRY["feature"] = code_agent.Agent() # 假设类型为feature的任务由Code Agent执行
AGENT_REGISTRY["bug"] = code_agent.Agent() # bug类型任务也交给Code Agent
AGENT_REGISTRY["task"] = code_agent.Agent() # 一般任务默认Code Agent
AGENT_REGISTRY["design"] = design_agent.Agent() # 设计类任务由Design Agent执行
AGENT_REGISTRY["default"] = code_agent.Agent() # 默认情况

def get_ready_tasks() -> List[Dict[str, Any]]:
    """
    调用 Beads 的 CLI 获取当前所有未阻塞的待执行任务列表。
    返回值为任务字典的列表，例如每个字典包含id、title、type、priority等字段。
    """
    try:
        # 调用 `bd ready --json` 获取 ready 状态任务的JSON列表
        result = subprocess.run(
            ["bd", "ready", "--json"], check=True,
            capture_output=True, text=True
        )
    except subprocess.CalledProcessError as e:
        print("Error: Failed to fetch tasks. Ensure `bd` (Beads CLI) is installed and initialized.")
        return []
    # 解析 JSON 输出为 Python 列表
    try:
        tasks = json.loads(result.stdout)
    except json.JSONDecodeError:
        print("Error: Failed to parse `bd ready` output.")
        tasks = []
    return tasks

def mark_task_completed(task_id: str):
    """
    将指定任务标记为完成（关闭任务）。使用 Beads CLI `bd close` 命令更新任务状态。
    """
    try:
        subprocess.run(
            ["bd", "close", task_id, "--reason", "Completed by orchestrator"],
            check=True, capture_output=True, text=True
        )
    
```

```

    )
    print(f"Task {task_id} marked as completed.")
except subprocess.CalledProcessError:
    print(f"Warning: Failed to close task {task_id} via bd CLI.")

def main():
    # 确保Beads已经初始化
    if not os.path.exists(".beads"):
        print("Beads not initialized. Initializing a new beads repository...")
        subprocess.run(["bd", "init"], check=True)
    print("Orchestrator started. Fetching tasks...")
    # 主循环：不断获取 ready 任务并执行，直到没有待执行任务
    while True:
        tasks = get_ready_tasks()
        if not tasks:
            print("No ready tasks. All done or waiting for dependencies.")
            break

        # 根据并发配置选择执行模式
        if MAX_CONCURRENT_AGENTS > 1:
            # 并发执行多个任务
            with ThreadPoolExecutor(max_workers=MAX_CONCURRENT_AGENTS) as executor:
                future_to_task = {}
                for task in tasks:
                    task_id = task.get("id")
                    task_title = task.get("title") or "<no title>"
                    task_type = task.get("type") or "default"
                    # 获取对应Agent（没有匹配类型则用"default"）
                    agent = AGENT_REGISTRY.get(task_type, AGENT_REGISTRY["default"])
                    print(f"Dispatching task {task_id} [{task_type}] to agent: {agent.name}")
                    # 提交Agent执行任务，将future与task映射以便后续处理
                    future = executor.submit(agent.run, task)
                    future_to_task[future] = task_id

                # 等待所有并发任务完成
                for future in as_completed(future_to_task):
                    t_id = future_to_task[future]
                    try:
                        result = future.result() # 获取Agent执行结果（如果有返回值）
                        print(f"Task {t_id} completed with result: {result}")
                    except Exception as exc:
                        print(f"Task {t_id} generated an exception: {exc}")
                    # 无论成功与否，尝试标记任务完成（由人工验证结果决定是否重新打开任务）
                    mark_task_completed(t_id)
            else:
                # 串行执行每个任务
                for task in tasks:
                    task_id = task.get("id")
                    task_type = task.get("type") or "default"
                    agent = AGENT_REGISTRY.get(task_type, AGENT_REGISTRY["default"])
                    print(f"Executing task {task_id} with agent: {agent.name}")

```

```

try:
    result = agent.run(task)
    print(f"Task {task_id} done. Result: {result}")
except Exception as exc:
    print(f"Error executing task {task_id}: {exc}")
    mark_task_completed(task_id)

# 循环继续，检查在执行过程中是否创建了新任务或仍有未完成任务
print("Checking for new ready tasks...\n")

if __name__ == "__main__":
    main()

```

`# AI多Agent编排系统 - 优化架构设计与实现`

### ## 架构概览

本项目基于 Python 实现一个多 Agent 协同工作的编排系统，利用 `**Kiro CLI**`、`**Beads**` 和 `**Agent SOP**` 等工具来组织 AI 代理的工作流程。优化后的架构将不同功能模块清晰分离，增强了系统的稳定性和可扩展性。

**核心思想：**由一个 Python 编写的 `orchestrator`（调度器）统一管理多个 AI Agent，每个 Agent 拥有独立的 SOP（标准操作流程）文件和技能（Skill）模块，以及各自关联的任务数据。调度器通过 Python SDK（而非手工终端命令）调用 Agent，串行或并发地调度任务执行，并使用 Beads 作为任务追踪与记忆机制，确保任务可以中断恢复、递归创建子任务等。

- **Agent SOP：**每个 Agent 的 SOP 定义了其执行特定任务的步骤流程，采用 Markdown 编写，包含任务目标、步骤和约束等信息【9†L311-L318】。SOP 相当于为 AI 代理设计的操作手册，使复杂的多步任务在不同 Agent 和团队之间可重复复用。
- **技能 (Skill)：**每个 Agent 具有一组技能模块，封装了该 Agent 可用的工具或操作（例如读取/写入文件、执行 Shell 命令等）。通过将技能与 Agent 解耦，每个 Agent 可以灵活定制其权限和功能，提升安全性和可控性。
- **调度器 (Orchestrator)：**作为系统的大脑，负责从任务列表中提取待办任务，分配给合适的 Agent 执行。调度器实现了稳定的管理逻辑，可根据配置选择串行或并发方式运行多个任务，并能够协调多个 Agent 间的协同（例如一个任务完成后触发另一 Agent 执行后续任务）。调度循环设计为可靠的事件驱动模型，支持自动恢复（在任务完成或被阻塞时动态调整）【21†L702-L709】。
- **Beads 任务管理：**本系统使用 Beads 作为任务的持久化存储和依赖管理工具【7†L77-L85】。Beads 将所有任务组织为带优先级和依赖关系的有向无环图 (DAG)，并通过 Git 将任务状态持久化存储在项目的 `.beads/issues.jsonl` 文件中【7†L124-L131】。每条任务（又称 `*bead*`）都有唯一 ID（如 `bd-123`）、状态（`open/in_progress/blocked/closed` 等）以及依赖情况。Beads 提供 `bd` CLI 工具，支持使用 `-json` 输出任务数据，方便我们的调度器程序解析和利用【7†L142-L150】。

通过上述架构，系统实现了以下优化目标：

- **清晰的模块分工：**每个 Agent 的 SOP、技能代码和任务数据彼此独立，存放于专属目录，便于团队协作和日后维护扩展。
- **稳定可控的调度：**调度器集中管理所有 Agent 的任务执行，确保执行顺序和并发度在掌控之中。调度器设计了串行和并发两种调度模式，可配置线程池大小控制并发 Agent 数量，支持多 Agent 同时运行。
- **规范的 SOP 编写：**约定了 SOP 文档的书写格式和规范（详见下文），包括统一的章节结构和约束标记，支持模块化和模板化生成 SOP，从而方便版本管理和知识复用。建议在 SOP 中明确使用 `**MUST**`（必须）、`**SHOULD**`（应该）等关键词注明约束要求，以确保执行的一致性【16†L1-

L4】。

- \*\*任务中断恢复与递归调用\*\*借助 Beads 的任务跟踪，未完成的任务状态会持久保存。当系统重启或中断后，调度器可通过 `\bd ready\` 自动检索待办任务继续执行，确保长周期任务不中断【7+L142-L150】。同时，Agent 在执行过程中若发现新的工作项，可动态创建子任务并以依赖关系链接到当前任务（使用 Beads 的 `*discovered-from*` 依赖类型【7+L169-L177】），调度器将识别这些关系，实现任务的递归调度和闭环管理。例如，测试Agent在执行测试用例时如果发现BUG，会创建一个新任务标记为Bug并将原测试任务标记为被阻塞（`blocked`），调度器检测到新的Bug任务（高优先级）已ready后，会自动启动专门的修复Agent处理该Bug；待Bug任务关闭后，原测试任务自动解除阻塞，调度器继续安排测试Agent完成后续工作【21+L812-L820】【21+L822-L830】。

接下来，我们将详细介绍项目的目录结构、核心模块的实现代码，以及使用方法和开发规范。

## ## 项目目录结构

```
``plaintext
ai_multi_agent_project/          # 项目根目录
├── orchestrator.py              # 调度器主程序
├── agents/                      # 所有 Agent 定义的模块目录
│   ├── __init__.py              # Agent 基类定义
│   ├── base_agent.py            # 示例 Agent：代码编写助手
│   └── code_agent/               # 示例 Agent：设计/规划助手
│       ├── __init__.py          # Code Agent 的 SOP 文件（标准操作流程）
│       ├── sop.md                # Code Agent 的技能函数定义
│       └── skills.py              # 示例 Agent：设计/规划助手
│   └── design_agent/
│       ├── __init__.py          # Design Agent 的 SOP 文件
│       ├── sop.md                # Design Agent 的技能函数定义
│       └── skills.py
└── .beads/                      # Beads 任务存储目录（Git仓库，用于任务持久化）
    └── issues.jsonl              # 任务数据主文件（JSONL格式，每行一个任务）
```

## 目录说明：

- `orchestrator.py`：核心调度脚本，负责读取任务、分配 Agent、执行任务及更新任务状态。
- `agents/base_agent.py`：定义所有具体 Agent 的基础类，封装公共逻辑，例如加载 SOP、执行任务的接口等。
- `agents/code_agent/` 和 `agents/design_agent/`：两个示例 Agent 的模块目录，每个 Agent 模块下都有：
  - `sop.md`：该 Agent 的 SOP 文件，采用 Markdown 编写，定义了此 Agent 执行任务的标准流程。
  - `skills.py`：该 Agent 可用的技能函数，通常是对底层工具的封装。例如 Code Agent 的技能包括读取/写入代码文件、运行测试用例等；Design Agent 则可能有读取文档、搜索资料等技能函数。
  - `__init__.py`：将目录作为 Python 包，使 orchestrator 能导入 Agent 模块。通常还在此定义 Agent 类（继承自 BaseAgent 并绑定对应的 sop 和 skills）。
- `.beads/`：Beads 的隐藏目录，存放任务数据。`issues.jsonl` 是主要的任务数据库文件，每行一个 JSON 记录任务详情（ID、标题、描述、优先级、状态、依赖关系等）<sup>3</sup>。此目录由 Beads 工具自动维护，一般不需手动编辑，但应纳入版本管理以保留任务进度历史。

整个项目遵循常规的 Python 项目命名规范，模块命名使用小写和下划线，目录层次清晰，方便进一步扩展新的 Agent 模块或其他功能。

## 核心模块实现

以下介绍本项目的核心代码，包括调度器和示例 Agent 的实现。代码均以代码块形式给出完整内容，并附带注释帮助理解。

### 调度器：orchestrator.py

调度器是系统的控制中枢。它通过周期性查询 Beads 获取当前所有“已就绪”（ready）的任务列表，然后按照预定策略将任务交由相应的 Agent 去执行。调度器确保任务执行的可控性：可根据需要串行地执行任务（一个接一个），或并发执行多个任务。同时，它负责在任务完成后更新任务状态（如关闭已完成的任务），并处理新发现的子任务。

下面是 orchestrator.py 的完整代码实现：

```
import os
import json
import subprocess
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import List, Dict, Any

# 配置：最大并发执行的 Agent 数量（线程数）。设置为1则串行执行。
MAX_CONCURRENT_AGENTS = 2

# 定义任务类型与 Agent 的映射。如无匹配则使用"default"对应的 Agent。
AGENT_REGISTRY: Dict[str, 'BaseAgent'] = {}

# 引入我们定义的 Agent 模块
from agents.base_agent import BaseAgent
from agents import code_agent, design_agent

# 初始化 Agent 实例并注册到映射表
AGENT_REGISTRY["feature"] = code_agent.Agent() # 类型为 feature 的任务由 Code Agent 执行
AGENT_REGISTRY["bug"] = code_agent.Agent() # bug 类型任务也交给 Code Agent
AGENT_REGISTRY["task"] = code_agent.Agent() # 一般任务默认由 Code Agent 处理
AGENT_REGISTRY["design"] = design_agent.Agent() # design 类型任务由 Design Agent 执行
AGENT_REGISTRY["default"] = code_agent.Agent() # 默认情况下使用 Code Agent

def get_ready_tasks() -> List[Dict[str, Any]]:
    """
    调用 Beads CLI 获取当前所有未被阻塞的待执行任务列表。
    返回任务字典列表，每个字典包含任务的 id、title、type、priority 等信息。
    """
    try:
        # 使用 `bd ready --json` 获取 ready 状态任务的 JSON 列表
        result = subprocess.run(
            ["bd", "ready", "--json"], check=True,
            capture_output=True, text=True
        )
    except subprocess.CalledProcessError:
        print("Error: 获取任务列表失败，请确认已安装并初始化 `bd` 工具。")
```



```

        return []
# 解析 JSON 输出为 Python 列表
try:
    tasks = json.loads(result.stdout)
except json.JSONDecodeError:
    print("Error: 解析 `bd ready` 输出失败。")
    tasks = []
return tasks

def mark_task_completed(task_id: str):
    """
    将指定任务标记为完成（关闭任务），调用 `bd close` 更新任务状态。
    """
    try:
        subprocess.run(
            ["bd", "close", task_id, "--reason", "Completed by orchestrator"],
            check=True, capture_output=True, text=True
        )
        print(f"Task {task_id} marked as completed.")
    except subprocess.CalledProcessError:
        print(f"Warning: Failed to close task {task_id} via bd CLI.")

def main():
    # 若未初始化 Beads 仓库则进行初始化（首次运行）
    if not os.path.exists(".beads"):
        print("Initializing Beads repository...")
        subprocess.run(["bd", "init"], check=True)
    print("Orchestrator started. Fetching tasks...")
    # 主循环：持续获取 ready 任务并执行，直到无任务可执行
    while True:
        tasks = get_ready_tasks()
        if not tasks:
            print("No ready tasks. All done or waiting for dependencies.")
            break

        if MAX_CONCURRENT_AGENTS > 1:
            # 并发执行多个任务
            with ThreadPoolExecutor(max_workers=MAX_CONCURRENT_AGENTS) as executor:
                future_to_task: Dict[Any, str] = {}
                for task in tasks:
                    task_id = task.get("id")
                    task_type = task.get("type") or "default"
                    agent = AGENT_REGISTRY.get(task_type, AGENT_REGISTRY["default"])
                    print(f"Dispatching task {task_id} [{task_type}] to agent: {agent.name}")
                    # 提交任务给线程池，由指定 Agent 执行
                    future = executor.submit(agent.run, task)
                    future_to_task[future] = task_id

            # 等待所有并发任务执行完成
            for future in as_completed(future_to_task):
                t_id = future_to_task[future]

```



```

        try:
            result = future.result() # 获取 Agent 执行结果
            print(f"Task {t_id} completed with result: {result}")
        except Exception as exc:
            print(f"Task {t_id} generated an exception: {exc}")
            # 无论成功与否，都尝试标记任务完成（必要时可人工复查后重新打开）
            mark_task_completed(t_id)

    else:
        # 串行逐个执行任务
        for task in tasks:
            task_id = task.get("id")
            task_type = task.get("type") or "default"
            agent = AGENT_REGISTRY.get(task_type, AGENT_REGISTRY["default"])
            print(f"Executing task {task_id} with agent: {agent.name}")
            try:
                result = agent.run(task)
                print(f"Task {task_id} done. Result: {result}")
            except Exception as exc:
                print(f"Error executing task {task_id}: {exc}")
                mark_task_completed(task_id)

        # 循环继续，检查执行过程中是否有新任务产生或剩余任务
        print("Checking for new ready tasks...\n")

if __name__ == "__main__":
    main()

```

上述代码实现中值得注意的几点：

- **任务获取：**函数 `get_ready_tasks()` 每次循环调用 `bd ready --json` 获取所有未被阻塞的任务列表。由于 Beads 每个命令均支持 JSON 输出，便于程序解析<sup>4</sup>。
- **任务分配：**通过 `AGENT_REGISTRY` 将任务类型（type）映射到具体的 Agent 实例。例如 type 为 "bug" 的任务会路由到相应的 Agent 处理。项目可以根据任务命名或类型设定映射规则，例如约定任务标题或类型包含特定关键词时选用特定 Agent<sup>11</sup>。
- **并发执行：**利用 `ThreadPoolExecutor` 实现多任务并行处理。通过设置 `MAX_CONCURRENT_AGENTS` 控制并发的线程数（Agent 数量）。如果将其设为1，则调度器按顺序串行执行任务；大于1则会并发提交任务执行，加快多任务场景下的处理速度。
- **结果处理：**无论任务执行成功与否，调度器都会调用 `mark_task_completed()` 尝试关闭任务（标记为完成）。这一步会调用 `bd close` 在 Beads 中更新任务状态为已完成，并添加简单的原因备注。这样设计可以防止任务卡住影响后续流程。若某任务执行失败，开发者可根据日志决定是否重新打开（reopen）任务或创建新任务。
- **循环与恢复：**调度器在每轮任务执行完毕后，会继续循环检查是否有新产生的 ready 任务。如果某个任务在执行过程中创建了后续子任务（例如 Agent 拆分出新任务），由于初始任务尚未关闭，新任务通常被标记为阻塞状态(discovered-from依赖)，不会立即出现在 ready 列表；一旦初始任务完成关闭，子任务就解除阻塞进入 ready 队列，下一轮循环将获取并执行它们。这种机制实现了任务的递归调度和自动恢复<sup>9</sup><sup>10</sup>。调度器循环在没有任何 ready 任务时退出，此时可能表示所有任务已完成，或剩余任务均在等待其它依赖完成。

## Agent 基类: agents/base\_agent.py

Agent 基类封装了各类 Agent 共有的功能，例如加载 SOP 文件、加载技能模块，以及定义统一的 `run()` 接口以执行任务。这样不同的 Agent 实现可以复用基类的逻辑，只需关注其特定的 SOP 内容和技能集合。

```
import os
import importlib

class BaseAgent:
    def __init__(self, name: str, sop_path: str, skills_module=None):
        self.name = name          # Agent 名称 (用于日志)
        self.sop_path = sop_path  # SOP 文件路径
        self.sop_content = self._load_sop()  # SOP 文件内容
        self.skills = {}
        # 加载技能函数 (如果提供了技能模块)
        if skills_module:
            # 将模块中可调用的属性收集为技能字典
            for attr_name in dir(skills_module):
                attr = getattr(skills_module, attr_name)
                if callable(attr):
                    self.skills[attr_name] = attr

    def _load_sop(self) -> str:
        """读取 SOP 文件内容"""
        try:
            with open(self.sop_path, 'r') as f:
                content = f.read()
            return content
        except FileNotFoundError:
            raise RuntimeError(f"SOP file not found: {self.sop_path}")

    def run(self, task: dict) -> str:
        """
        执行指定任务。这里会按照 SOP 指引实际调用AI模型或工具完成任务。
        返回任务执行结果摘要或状态。
        """
        task_id = task.get("id", "")
        task_title = task.get("title", "")
        print(f"[{self.name}] 开始执行任务 {task_id}: {task_title}")
        # TODO: 将 task_title 等作为输入，结合 SOP 内容，引导 AI 完成任务。
        # 提示：实际实现中，可利用 Agent SOP 的 Python SDK 调用模型执行。
        # 例如使用 Strands 的 Agent 类:
        # from strands import Agent as StrandsAgent
        # agent_instance = StrandsAgent(system_prompt=self.sop_content,
        tools=list(self.skills.values()))
        # agent_instance(f"任务: {task_title}\n请按照SOP步骤完成。")
        # 为简单起见，这里仅模拟执行过程:
        result_summary = f"[{self.name}] 完成了任务。"
        print(f"[{self.name}] 已完成任务 {task_id}")
        return result_summary
```

以上 `BaseAgent` 做了如下工作：

- **SOP加载：**在初始化时，从给定路径读取 SOP 文件全文并存储在 `self.sop_content`。这样 Agent 后续执行任务时可以随时引用 SOP 的指令集。如果文件不存在，会抛出异常提醒。
- **技能加载：**如果提供了技能模块，`BaseAgent` 会反射获取该模块中所有可调用对象，将其加入 `self.skills` 字典。这样 Agent 在执行过程中可以访问命名的技能函数。例如 `self.skills["open_file"]` 即对应打开文件的函数实现。
- **执行接口：**`run(self, task)` 方法是调度器调用的核心接口。它接受一个任务描述（字典），提取其中的任务ID和标题等信息用于日志。实际执行中，该方法应当结合 SOP 内容和任务详情，驱动 AI 模型一步步完成 SOP 中定义的步骤。理想情况下，我们可以使用现成的 Agent SDK，例如 Strands 提供的 Agent 类，将 SOP Markdown 作为系统提示，技能函数作为可用工具，然后调用模型开始执行<sup>12</sup>。在此范例中，我们以伪代码和日志打印模拟了执行过程，并返回一个简单的结果摘要字符串。

**说明：**实际实现时，`run()` 方法会与底层的大模型（LLM）交互，例如通过 OpenAI/Anthropic 的 API，或者调用 Kiro/Strands 的 Python SDK 来运行 Agent SOP。确保在调用前已配置好必要的 API 密钥或 Kiro CLI 环境。我们的框架将 SOP 内容提供给模型，使 AI 代理严格按照 SOP 步骤工作。在 Agent 执行过程中，若需要使用某项技能（如读写文件），AI 可以在对话中请求，开发者可拦截该请求并调用 `self.skills` 中对应函数，或预先通过 SDK 将这些函数注册为工具供模型直接调用。

## Code Agent 实现： `agents/code_agent/`

**Code Agent** 用于处理编码相关的任务（例如实现新功能、修复代码缺陷等）。它的 SOP 描述了如何以TDD风格完成编码任务，技能模块则提供读写代码、运行测试等能力。下面分别展示其技能定义和 SOP 文件内容，以及 Agent 类的定义。

**文件：** `agents/code_agent/skills.py`

Code Agent 的技能函数，提供文件操作和测试执行等能力。

```
import subprocess

def open_file(filepath: str) -> str:
    """打开指定文件并返回其内容。"""
    try:
        with open(filepath, 'r') as f:
            content = f.read()
        return content
    except Exception as e:
        return f"Error opening file {filepath}: {e}"

def run_tests(command: str = "pytest") -> str:
    """执行测试命令，返回测试输出结果。"""
    result = subprocess.run(command, shell=True, capture_output=True, text=True)
    output = result.stdout.strip()
    if result.returncode == 0:
        return f"Tests passed.\n{output}"
    else:
        return f"Tests failed!\n{output}\n{result.stderr}"

def create_task(title: str, task_type: str = "task", priority: int = 2) -> str:
```

```

"""创建一个新的 Beads 任务，返回新任务的ID。"""
try:
    result = subprocess.run(
        ["bd", "create", title, "-t", task_type, "-p", str(priority)],
        check=True, capture_output=True, text=True
    )
    # 假设 stdout 包含新建任务的 ID 和标题
    return result.stdout.strip()
except subprocess.CalledProcessError as e:
    return f"Failed to create task: {e}"

```

以上技能函数分别实现：

- `open_file` : 打开文件并读取内容（供 Agent 查看现有代码或文档）。
- `run_tests` : 在 Shell 中运行测试命令（默认 `pytest`），返回测试输出，指示测试是否通过。
- `create_task` : 调用 `bd create` 创建新任务，用于 Agent 将复杂任务拆解为子任务记录到 Beads。例如，Agent 可在计划阶段调用此函数新增子任务，并通过 Beads 将其与当前任务关联（如添加 `discovered-from` 依赖）<sup>13</sup>。

文件： `agents/code_agent/sop.md`

Code Agent 的 SOP 文件，定义了代码任务的标准操作流程。

## # Code Agent SOP

### ## 概述

该 SOP 指导 Code Agent 使用测试驱动开发（TDD）方法，根据任务描述编写代码实现。

### ## 参数

- **task\_description** (必填)：要完成的代码任务描述。
- **mode** (可选，默认： "interactive")：工作模式， "interactive" 表示交互式逐步实施， "fsc" 表示全自主编程模式。

### ## 步骤

#### ### 1. 准备阶段

##### \*\*约束:\*\*

- 必须确保开发环境已配置（例如依赖安装、项目结构就绪）。
- 必须充分理解任务描述和验收标准，并记录在案。

#### ### 2. 计划任务

##### \*\*约束:\*\*

- 应列出实现任务的分解步骤和方案。
- 可以使用 `'bd create'` 创建子任务来跟踪这些步骤<sup>13</sup>。

#### ### 3. 编码实现

##### \*\*约束:\*\*

- 必须根据计划逐步编写代码，每完成一个子步骤应运行对应测试确保通过。
- 如遇障碍，可以创建新的 Beads 任务记录发现的问题（使用 `*discovered-from*` 链接回当前任务）。

#### ### 4. 测试与验证

##### \*\*约束:\*\*

- 必须运行所有相关测试，确保新代码通过测试。
- 如果测试未通过，应该调试并修复代码后重跑测试，直至全部通过。

### ### 5. 完成收尾

#### \*\*约束:\*\*

- 完成任务后，应更新 Beads 中任务的备注和状态（如添加解决方案摘要），然后关闭任务。
- 应生成简要的变更日志或文档，说明实现的功能和注意事项。

该 SOP 文件结构清晰，包含概述、参数定义，以及按步骤编号的操作指南。每个步骤列出了具体要求，并使用“必须(MUST)/应该(SHOULD)”等措辞强调了约束条件，使AI代理在执行过程中严格遵循这些规则<sup>8</sup>。在步骤2和步骤3，还示例性地提到了使用 Beads 记录子任务和问题的做法，体现了任务拆解和追踪的规范。

文件：agents/code\_agent/\_\_init\_\_.py

定义 Code Agent 的 Agent 类，继承自 BaseAgent 并指定专属的 SOP 和技能集。

```
import os
from agents.base_agent import BaseAgent
from . import skills

class Agent(BaseAgent):
    def __init__(self):
        sop_file = os.path.join(os.path.dirname(__file__), "sop.md")
        super().__init__(name="CodeAgent", sop_path=sop_file, skills_module=skills)
```

这里我们通过继承 BaseAgent 来创建具体的 Agent 类。在构造函数中，组装传递给基类的参数：

- name : 设置 Agent 名称为 "CodeAgent"。
- sop\_path : 指向 Code Agent 自身的 SOP 文件路径。
- skills\_module : 引用该模块下的 skills 模块，以便 BaseAgent 可以加载所有技能函数。

完成上述设置后，CodeAgent 就准备就绪，可以被调度器实例化并调用其 run() 方法执行任务了。

### Design Agent 实现：agents/design\_agent/

**Design Agent** 用于处理设计构思、规划和文档编写等任务。它的 SOP 强调分析规划和文档输出，技能上可能涉及搜索资料等。其结构与 Code Agent 类似，以下给出主要文件内容。

文件：agents/design\_agent/skills.py

Design Agent 的技能函数，提供文档查找等能力。

```
def search_docs(keyword: str) -> str:
    """ (示例) 搜索内部知识库文档，返回简要结果。 """
    # 实际实现可调用API或数据库，此处简单模拟
    print(f"[DesignAgent] Searching docs for: {keyword}")
    return f"Results for '{keyword}': ... (dummy data)"
```

(Design Agent 的技能相对简单，此处仅示范一个搜索功能。实际情况下，可扩展为调用企业内部Wiki或互联网搜索的接口。)

文件: `agents/design_agent/sop.md`

Design Agent 的 SOP 文件，定义了设计/规划任务的标准流程。

## # Design Agent SOP

### ## 概述

该 SOP 指导 Design Agent 根据任务要求进行技术设计、方案规划及文档编写。

### ## 参数

- **task\_description** (必填): 设计任务的简要描述。

### ## 步骤

#### ### 1. 需求分析

##### \*\*约束:\*\*

- 必须充分理解任务背景和需求，明确目标和范围。

#### ### 2. 拟定方案

##### \*\*约束:\*\*

- 应提出至少两个可行方案并比较优劣，选择最佳方案。
- 如任务较大，可将其拆解成子任务记录在 Beads 中以便跟踪。

#### ### 3. 文档撰写

##### \*\*约束:\*\*

- 必须根据选定方案编写详细的设计文档，包括架构图、流程图等必要内容。
- 文档应清晰组织，涵盖关键决策及理由。

#### ### 4. 评审完善

##### \*\*约束:\*\*

- 应自我检查并通过同事评审文档，修正其中的问题。
- 确保文档足以指导实现，无重大全球遗漏。

#### ### 5. 提交成果

##### \*\*约束:\*\*

- 完成设计后，在 Beads 中更新任务状态和备注，总结设计输出。
- 通知相关人员评审通过，设计定稿。

文件: `agents/design_agent/__init__.py`

定义 Design Agent 的 Agent 类。

```
import os
from agents.base_agent import BaseAgent
from . import skills

class Agent(BaseAgent):
    def __init__(self):
        sop_file = os.path.join(os.path.dirname(__file__), "sop.md")
        super().__init__(name="DesignAgent", sop_path=sop_file, skills_module=skills)
```

Design Agent 的实现与 Code Agent 十分类似，只是名称、SOP路径和技能模块不同。在实际应用中，还可以根据需要进行扩充 Design Agent 的技能函数，例如整合绘图工具生成架构图、集成第三方API获取资料等等。

通过上述两个示例 Agent，可以看出每个 Agent 的代码组织遵循统一模式：**SOP文件**定义流程、**skills模块**提供能力、**Agent类**结合二者继承基类。在新增 Agent 时，可复用这一模式快速扩展。

## 使用说明与开发规范

本节介绍如何配置运行本项目，以及未来扩展新 Agent、编写 SOP 和管理 Beads 任务的建议规范。

### 环境准备与运行

1. **环境依赖**：确保已安装 Python，以及 Kiro CLI 和 Beads 工具。Kiro CLI 提供底层AI代理能力，Beads 提供任务追踪。还需要配置好AI模型的访问，如 OpenAI API Key 或 Anthropic Key（如果 Kiro CLI 已集成，可参考其文档配置）。
2. **安装 Beads**：可以通过 Homebrew 或 pip 安装 Beads MCP。例如：

```
brew install steveyegge/beta/beads # 或使用 pip: pip install beads-mcp
```

安装完成后，在项目目录下执行 `bd init` 初始化一个 Beads 仓库。这会创建 `.beads/` 目录和初始的 `issues.jsonl` 文件。

3. **添加初始任务**：使用 `bd create` 添加一些测试任务。例如：

```
bd create "Demo: 实现用户登录功能" -t feature -p 1
bd create "Design: 设计数据库架构" -t design -p 2
bd create "Bug: 修复登录页面崩溃问题" -t bug -p 0
```

以上命令创建了三个任务：一个新功能(feature)任务、一个设计任务、一个紧急Bug任务。我们使用 `-t` 指定任务类型，`-p` 指定优先级（0为最高）。根据我们的 `AGENT_REGISTRY`，feature和bug类型任务将由 CodeAgent 执行，design类型任务由 DesignAgent 执行。

4. **运行调度器**：在项目根目录运行：

```
python orchestrator.py
```

调度器启动后，会自动调用 `bd ready --json` 获取当前所有未阻塞的任务，然后依次分配给Agent执行。控制台会输出日志，指示哪些任务被分配给哪个Agent、执行结果如何等。

5. 由于我们设置了 `MAX_CONCURRENT_AGENTS = 2`，调度器将同时执行两个任务。例如，上述示例中优先级最高的Bug任务和Feature任务可能被并发执行，而优先级较低的Design任务将在下一轮执行（或等前面的完成视具体实现）。



6. 调度过程中，如果某任务因为依赖未满足而处于blocked状态（不在 ready 列表），调度器会跳过它。等依赖解除后（例如相关任务完成关闭），该任务才会出现在 ready 列表中，被后续循环处理。
7. 所有任务执行完成并被标记关闭后，调度器会输出"No ready tasks. All done."并结束运行。
8. **查看任务状态：**可随时使用 Beads 命令查看任务进展：

9. `bd list --status open` 列出所有未完成任务。
10. `bd show <任务ID>` 查看特定任务详情（描述、备注、依赖等）。
11. `bd stats --json` 获取任务统计信息（多少已完成、阻塞、中断等）。

在调度器运行期间，Agent 会按照其 SOP 逐步执行任务。例如，当 CodeAgent 执行 "Demo: 实现用户登录功能" 时，它将依据 SOP 先规划步骤，可能创建子任务（例如“实现后端登录API”等），这些子任务最初会标记为发现自父任务而阻塞<sup>13</sup>。当父任务完成关闭后，这些子任务就成为新的 ready 任务，调度器将在后续循环自动调度它们，从而实现自动的递归任务执行。整个过程无需人工干预，依赖 Beads 的任务依赖机制来串联<sup>9</sup><sup>10</sup>。

## 开发与扩展指南

### 1. 新增 Agent 模块：如果需要添加新的 Agent，例如处理部署任务的Agent：

- 新建目录 `agents/deploy_agent/`，编写其 `sop.md` 和 `skills.py`，定义部署流程的 SOP 以及相关技能函数（比如执行部署脚本、监控服务状态等）。
- 在 `agents/deploy_agent/__init__.py` 中创建 `Agent` 类继承自 `BaseAgent`，传入该Agent名称、SOP 路径和技能模块。
- 修改 `orchestrator.py` 中的 `AGENT_REGISTRY`，增加相应的任务类型映射到 `deploy_agent.Agent()`。同时制定任务命名或类型规范，例如约定所有部署相关任务类型为 "deploy"，这样 Beads 中创建任务时使用 `-t deploy` 即可路由到该 Agent。

### 2. SOP 编写规范：遵循统一的格式书写 SOP：

- **文件命名：**使用有意义的文件名，如 `sop.md` 放在各Agent子目录下，或根据用途命名如 `deploy_sop.md`。使用 Markdown 格式书写方便阅读和版本控制。
- **章节结构：**通常包括“# 标题（SOP名称）”、“## 概述”、“## 参数”（如有参数化需求）、“## 步骤”等部分。步骤使用有序标题（### 1、### 2 ...）分节，每步下列出具体指引。
- **约束措辞：**在步骤的说明中使用**必须(MUST)**、**应该(SHOULD)**、**可以(MAY)**等词语明确要求的强制性和可选项<sup>8</sup>。这有助于AI严格遵守重要步骤。例如：“必须编写测试用例”表示此步骤不可跳过；“可以优化代码结构”表示允许的改进但非强制。
- **示例和模板：**针对常见模式的SOP，可以制作模板。例如编码类任务的 SOP 模板、文档类任务的 SOP 模板。编写新 SOP 时可从模板复制骨架并填写具体内容，确保一致性。利用版本控制跟踪 SOP 变更，必要时在文件头部增加版本号或日期注释。
- **调试与完善：**由于 SOP 是给AI看的“剧本”，实际执行效果需要观察AI的行为来调整。如果发现某些步骤AI经常误解或跳过，可在SOP中增加提示或拆分步骤。维护一套完善的SOP有助于不同Agent和不同版本的模型都能稳定按流程办事。

### 3. 技能函数规范：开发技能模块时请注意：

- **单一职责：**每个技能函数尽量执行一个明确动作，例如读取文件、调用API等。不要在一个函数内混合过多操作，便于AI调用和开发者调试。
- **错误处理：**技能函数应考虑可能的失败情况，做好异常捕获并返回友好的错误信息。例如文件不存在时返回明确的提示。这样AI得到反馈后可以据此决定后续行动（比如提醒用户文件未找到）。

- **与AI集成：**如果使用 Kiro/Strands 等SDK，可将这些函数注册为AI可调用的工具。也可以采用简单的方案：让AI以特定格式请求调用某技能，再由调度器捕获并调用相应函数，将结果插入AI对话。这部分机制可依据项目需求实现。
- **按需加载：**对于有大量技能的Agent，可采用延迟加载策略以节省上下文开销。例如 BaseAgent 可以改进为在 `run()` 开始前根据任务内容选择性地加载某些技能 <sup>14</sup> <sup>15</sup>。不过在当前实现中，技能数量有限，直接加载全部也问题不大。

#### 4. Beads 任务管理约定：为了充分发挥 Beads 在多Agent协作中的作用，推荐遵循以下约定：

- **命名和类型：**制定任务命名和类型分配规则，使调度器能精准路由任务 <sup>11</sup>。例如：
- 以“Bug:”开头的任务且类型标记为 `bug`，由Bug修复类Agent处理。
- 标题含“Design”且类型为 `design` 的任务，由DesignAgent处理。
- 长期可以在项目文档中维护一份任务命名/类型约定说明，团队按此规范创建任务以减少歧义。
- **任务拆解：**鼓励Agent将复杂任务拆分。利用 Beads 的依赖关系（如 `blocked by` 或 `discovered-from`）建立父子任务链路。当Agent发现额外工作时，创建新任务并正确设置依赖，可以让调度器自动处理协调，无需显式通信开销，Beads 将自动驱动任务流转 <sup>16</sup> <sup>17</sup>。
- **任务备注 (notes)：**Agent 完成任务后，应使用 `bd update --notes` 添加关键信息到任务备注。例如实现了什么、遇到哪些决定。这样即使任务关闭，这些上下文也持久保存，供日后查询 <sup>18</sup> <sup>19</sup>。我们的 SOP 已经在最后一步提醒Agent更新任务备注，开发者也应在Agent实现中确保执行该动作。
- **状态管理：**充分利用 Beads 的状态字段，例如在Agent开始处理任务时将其标记为 `in_progress`（进行中），阻塞时标记 `blocked` 等，调度器可以据此决定是否跳过或如何处理。当前实现中，我们简化处理，直接根据 `ready` 列表调度，但未来可以拓展调度器更智能地查看任务状态和依赖进行决策。
- **持久化与协作：**由于 `.beads/issues.jsonl` 由Git维护，可将其纳入仓库，团队协作时经常同步（git pull/push）该文件，使各人环境下的Agent共享同一任务队列和状态。这相当于一个轻量的分布式任务看板，结合自动化Agent极大提升开发效率 <sup>20</sup> <sup>21</sup>。

#### 5. 测试与调优：在引入真实AI模型执行之前，建议对 orchestrator 和 Agent 逻辑进行本地测试：

- 使用简单的任务和技能函数中打印输出的方式，验证调度器正确分配了任务，Agent 正确调用了技能并遵循了 SOP 流程。
- 针对常见场景（例如任务无依赖直接完成、有依赖的任务链、中途中断恢复等）进行演练，确保系统行为符合预期。利用 Beads 提供的统计和查询命令检查任务状态转变是否正确。
- 逐步引入AI模型，让Agent实际产出内容。这时需要仔细观察AI执行每一步是否符合 SOP，如有偏差，考虑调整 SOP 文本措辞或增加约束，或者在 Agent 实现中加强对AI输出的检查。
- 监控运行性能和开销。对于并发数的选择要考虑AI API速率限制和本地资源。Mac终端环境下并发运行多个Agent通常没有问题，但如果每个Agent都调用大型模型，可以酌情降低并发以避免占满带宽或导致响应延迟。

按照以上说明配置和开发，本项目应当可以在本地顺利运行，实现多Agent协同完成复杂任务的能力。在实际使用中，持续迭代优化各Agent的 SOP 和技能，将逐步提高整个系统的可靠性和智能协作水平。通过清晰的架构设计和规范，本项目为后续扩展打下了良好基础，开发者无需参考其他文档即可理解和使用。

---

<sup>1</sup> <sup>8</sup> <sup>12</sup> GitHub - strands-agents/agent-sop: Natural language workflows that enable AI agents to perform complex, multi-step tasks with consistency and reliability.  
<https://github.com/strands-agents/agent-sop>

<sup>2</sup> <sup>3</sup> <sup>4</sup> <sup>7</sup> <sup>20</sup> <sup>21</sup> Beads: Distributed Task Management for AI Agents | Peter Warnock  
<https://peterwarnock.com/blog/posts/beads-distributed-task-management/>

<sup>5</sup> Introducing Beads: A coding agent memory system | by Steve Yegge  
<https://steve-yegge.medium.com/introducing-beads-a-coding-agent-memory-system-637d7d92514a>

6 A Coding Agent Framework with Memory and Issue Tracking ...

<https://levelup.gitconnected.com/a-coding-agent-framework-with-memory-and-issue-tracking-combined-b75122828ee1>

9 10 11 14 15 16 17 projectsummary.md

[file:///file\\_00000000e8c471fd9988c75f75025564](file:///file_00000000e8c471fd9988c75f75025564)

13 18 19 bd-issue-tracking - Claude Skills

<https://claude-plugins.dev/skills/@blueplane-ai/bp-telemetry-core/beads>